

PROJECT REPORT

TITLE: Design and Implementation of a Database Management System for Vehicle Service Centre Management System



NAME: SAHITYA KORRAPATI

ROLL NO: 25WU0102234

COURSE NAME: Database Management Systems

FACULTY NAME: Dr Kiran Mayee Adavala

ACADEMIC YEAR: 2025-2029

ABSTRACT

The Vehicle Service Centre Management System is a database management system designed to efficiently manage and organize the operations of a vehicle service centre. The system maintains important information related to customers, vehicles, mechanics, service bookings, job cards, spare parts, invoices, and payments in a structured relational database. It uses primary and foreign keys to establish relationships between entities and maintain data integrity.

The project provides a web-based user interface connected to a MySQL database through a Node.js and Express.js backend. The interface allows users to perform essential database operations such as inserting, viewing, and deleting customer records, with changes being directly reflected in the database. The system aims to reduce manual record management, improve data accessibility, and provide an organized approach to handling vehicle service centre operations.

Overall, the project demonstrates the practical application of database design, relational models, SQL, normalization, database connectivity, and web-based CRUD operations in developing a real-world management system.

INTRODUCTION

- The Vehicle Service Centre Management System is a database-based application developed to manage and organize the various activities of a vehicle service centre. A service centre handles a large amount of information, including customer details, vehicle information, mechanics, service bookings, job cards, spare parts, invoices, and payments. Managing these records manually can make it difficult to maintain accurate and easily accessible information.
- The proposed system uses a relational database to store and manage these records systematically. The database consists of interconnected entities such as Customer, Vehicle, Mechanic, Service Booking, Job Card, Spare Part, Invoice, and Payment. Primary keys and foreign keys are used to establish relationships between the tables and maintain data integrity.
- The project also includes a web-based user interface connected to the MySQL database using Node.js and Express.js. The interface allows users to perform operations such as adding, viewing, and deleting customer records. This demonstrates how a database can be integrated with a user-friendly application for real-world service-centre management.

Problem Statement

- **Vehicle** service centres need to manage a large amount of customer and service-related data on a regular basis. When such information is handled manually or without a properly structured database, it can lead to data duplication, difficulty in retrieving records, inaccurate information, and inefficient management of service operations.
- There is therefore a need for a centralized database system that can store customer, vehicle, mechanic, booking, job card, spare part, invoice, and payment information in a structured manner. The system should maintain relationships between these entities while ensuring data consistency and integrity.
- The Vehicle Service Centre Management System aims to address these problems by providing a relational database and web-based interface through which service-centre records can be efficiently stored, retrieved, and managed.

Objectives

The main objectives of the Vehicle Service Centre Management System are:

- To develop a centralized database for storing and managing vehicle service centre information.
- To maintain customer and vehicle records in a structured and easily accessible manner.
- To manage service bookings and job cards efficiently.
- To maintain mechanic information and their service-related details.
- To manage spare-part information, including stock quantity, price, and supplier details.
- To manage invoices and payment records associated with completed services.
- To maintain data integrity and consistency using primary keys, foreign keys, constraints, and normalization.
- To provide a user-friendly web interface connected to the MySQL database.
- To implement database operations through the UI, including insertion, viewing, and deletion of customer records.
- To reduce manual record management and improve the efficiency of service-centre operations.

Scope of the Project

The scope of the Vehicle Service Centre Management System includes the design and implementation of a relational database for managing the major operations of a vehicle service centre.

The system covers:

- Customer Management – storing customer ID, name, phone number, email, and address.
- Vehicle Management – maintaining vehicle registration, brand, model, year, type, and associated customer.
- Mechanic Management – storing mechanic details, specialization, contact information, and availability status.
- Service Booking Management – recording booking dates, preferred service dates, service types, and booking status.
- Job Card Management – maintaining service descriptions, start/end dates, assigned mechanics, and job status.
- Spare Part Management – tracking spare parts, prices, stock quantities, and suppliers.
- Invoice Management – maintaining invoice details, total amounts, status, and associated customers/job cards.
- Payment Management – recording payment dates, amounts, payment methods, transaction references, and payment status.
- Web-based Database Interaction – providing a user interface connected to MySQL through Node.js and Express.js for performing database operations.

Software and Hardware Requirements

The Vehicle Service Centre Management System requires a basic computer system with suitable software tools for development, database management, and application execution. The project can be developed and executed using the following configuration:

Software Requirements

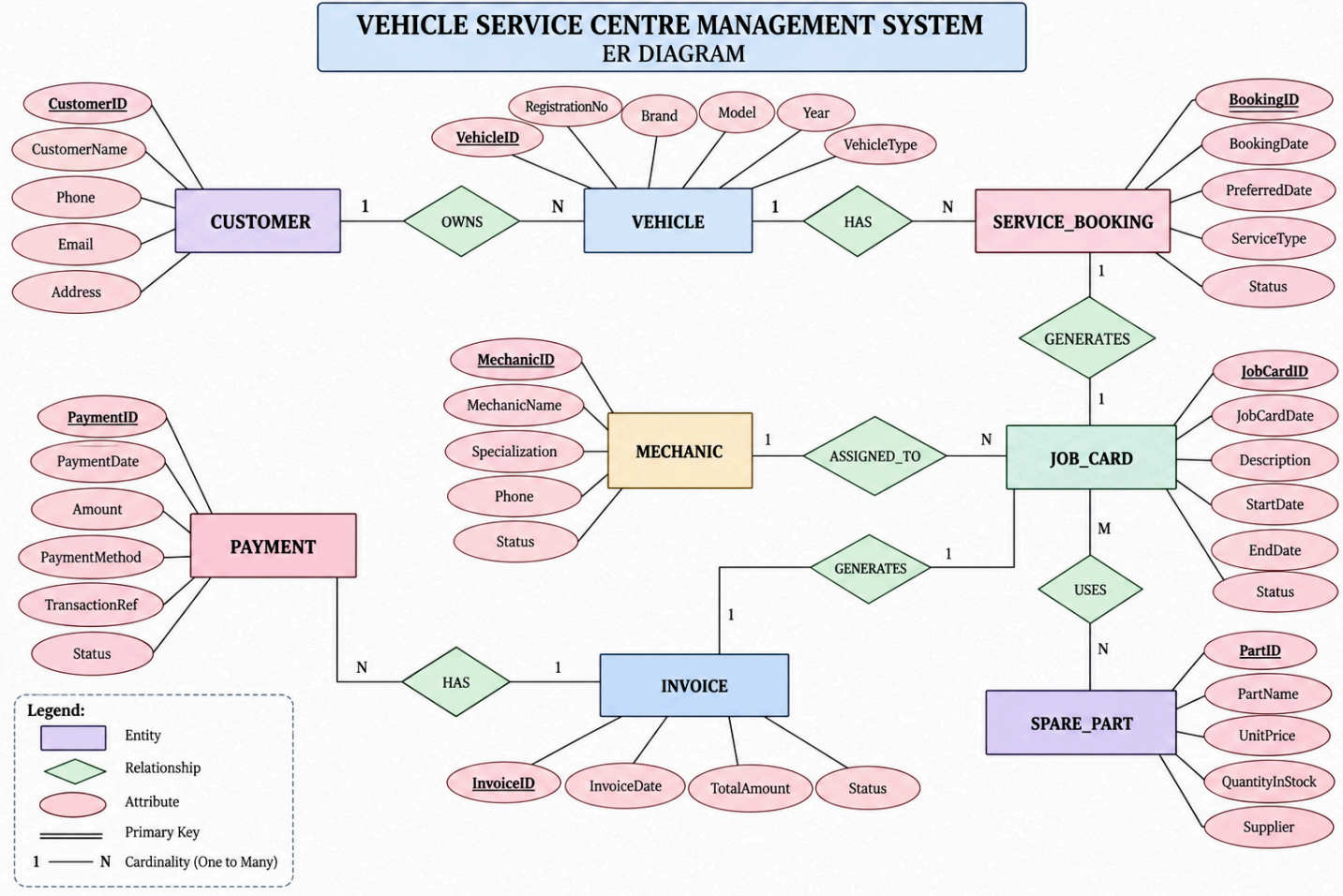
- Operating System: Windows 10/11 or equivalent
- Visual Studio Code: For writing and managing the project source code
- Node.js: For running the backend server
- Express.js: For developing backend APIs and connecting the UI with the database
- MySQL: Relational database management system
- Aiven MySQL: Cloud-hosted MySQL database used for storing project data
- HTML5 & CSS3: For designing the web-based user interface
- JavaScript: For implementing frontend functionality and database operations through APIs
- MySQL2: Node.js package used for connecting the application with MySQL
- Web Browser: Google Chrome, Microsoft Edge, or any modern browser
- Internet Connection: Required for accessing the Aiven cloud database

Hardware Requirements

- Processor: Intel Core i3 or equivalent and above
- RAM: Minimum 4 GB, preferably 8 GB
- Storage: At least 10 GB of available storage
- Display: Standard laptop/desktop display
- Keyboard and Mouse: For development and system interaction
- Internet Connectivity: Required for communication with the cloud-hosted MySQL database

Overall, the system requires a standard computer with a modern web browser, Node.js development environment, and internet connectivity to run the application and access the MySQL database hosted on Aiven.

ER DIAGRAM



Relational Schema and Normalization

Relational Schema

The project consists of 8 relational tables:

- Customer(CustomerID, CustomerName, Phone, Email, Address)
- Vehicle(VehicleID, RegistrationNo, Brand, Model, VehicleYear, VehicleType, CustomerID)
- Mechanic(MechanicID, MechanicName, Specialization, Phone, Status)
- Service_Booking(BookingID, BookingDate, PreferredDate, ServiceType, Status, CustomerID, VehicleID)
- Job_Card(JobCardID, JobCardDate, Description, StartDate, EndDate, Status, BookingID, MechanicID)
- Spare_Part(PartID, PartName, UnitPrice, QuantityInStock, Supplier)
- Invoice(InvoiceID, InvoiceDate, TotalAmount, Status, JobCardID, CustomerID)
- Payment(PaymentID, PaymentDate, Amount, PaymentMethod, TransactionRef, Status, InvoiceID)
- Primary keys uniquely identify records, while foreign keys establish relationships between the tables.

Normalization

The database is normalized up to Third Normal Form (3NF).

- 1NF: All attributes contain atomic values and each table has a primary key.
- 2NF: There are no partial dependencies; non-key attributes depend on the complete primary key.
- 3NF: There are no transitive dependencies, and related information is separated into different tables.

Result: Normalization reduces data redundancy, prevents update anomalies, and maintains data integrity in the Vehicle Service Centre Management System.

DATA DICTONARY

Table	Primary Key (PK)	Foreign Key (FK)
Customer	CustomerID	—
Vehicle	VehicleID	CustomerID
Mechanic	MechanicID	—
Service Booking	BookingID	CustomerID, VehicleID
Job Card	JobCardID	BookingID, MechanicID
Spare Part	PartID	—
Invoice	InvoiceID	JobCardID, CustomerID
Payment	PaymentID	InvoiceID

DDL Commands

```
CREATE TABLE Customer (  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(100) NOT NULL,  
    Phone VARCHAR(15) UNIQUE NOT NULL,  
    Email VARCHAR(100),  
    Address VARCHAR(200)  
);
```

OUTPUT

CustomerID	CustomerName	Phone	Email	Address
1	Rahul Sharma	9876543210	rahul@gmail.com	Hyderabad
2	Ananya Reddy	9123456789	ananya@gmail.com	Secunderabad
3	Arjun Kumar	9988776655	arjun@gmail.com	Hyderabad

DML Commands

```
INSERT INTO Customer (CustomerID, CustomerName, Phone, Email, Address) VALUES
(1, 'Rahul Sharma', '9876543210', 'rahul@gmail.com', 'Hyderabad'),
(2, 'Ananya Reddy', '9123456789', 'ananya@gmail.com', 'Secunderabad'),
(3, 'Arjun Kumar', '9988776655', 'arjun@gmail.com', 'Hyderabad');
```

OUTPUT:

CustomerID	CustomerName	Phone	Email	Address
1	Rahul Sharma	9876543210	rahul@gmail.com	Hyderabad
2	Ananya Reddy	9123456789	ananya@gmail.com	Secunderabad
3	Arjun Kumar	9988776655	arjun@gmail.com	Hyderabad

Query given during presentation-II

The given query was to display names of customers related to engine services.

```
-- QUERY EXECUTION:  
-- DISPLAY NAMES OF CUSTOMERS RELATED TO ENGINE SERVICES
```

```
SELECT Customer.CustomerName  
FROM Customer  
JOIN Vehicle  
| | ON Customer.CustomerID = Vehicle.CustomerID  
JOIN Service_Booking  
| | ON Vehicle.VehicleID = Service_Booking.VehicleID  
JOIN Job_Card  
| | ON Service_Booking.BookingID = Job_Card.BookingID  
WHERE Job_Card.Description LIKE '%Engine%';
```

OUTPUT

```
+-----+  
| CustomerName |  
+-----+  
| Rahul Sharma |  
| Ananya Reddy |  
| Arjun Kumar  |  
+-----+
```

UI DESIGN AND SCREENSHOTS

UI DESIGN

Vehicle Service Centre Management System

Customer Management Module

Add Customer

Customer ID

Customer Name

Phone

Email

Address

Add Customer

View Customers

Delete Customer

Enter Customer ID

Delete Customer

VIEWING OF RECORDS

Address

Add Customer

View Customers

Delete Customer

Enter Customer ID

Delete Customer

Customer Records

Customer ID	Name	Phone	Email	Address
1	Rahul Sharma	9876543210	rahul@gmail.com	Hyderabad
2	Ananya Reddy	9123456789	ananya@gmail.com	Secunderabad
3	Arjun Kumar	9988776655	arjun@gmail.com	Hyderabad

INSERTION OF RECORDS

Vehicle Service Centre Management System

Customer Management Module

Add Customer

Customer ID

11

Customer Name

sara

Phone

9000044444

Email

sara@gmail.com

Address

Hyd

Add Customer

View Customers

Delete Customer

Enter Customer ID

Delete Customer

AFTER:

Add Customer

View Customers

Delete Customer

Enter Customer ID

Delete Customer

Customer Records

Customer ID	Name	Phone	Email	Address
1	Rahul Sharma	9876543210	rahul@gmail.com	Hyderabad
2	Ananya Reddy	9123456789	ananya@gmail.com	Secunderabad
3	Arjun Kumar	9988776655	arjun@gmail.com	Hyderabad
11	sara	9000044444	sara@gmail.com	Hyd

DELETION OF RECORDS

BEFORE:

Add Customer

View Customers

Delete Customer

11

Delete Customer

Customer Records

Customer ID	Name	Phone	Email	Address
1	Rahul Sharma	9876543210	rahul@gmail.com	Hyderabad
2	Ananya Reddy	9123456789	ananya@gmail.com	Secunderabad
3	Arjun Kumar	9988776655	arjun@gmail.com	Hyderabad
11	sara	9000044444	sara@gmail.com	Hyd

AFTER:

Add Customer

View Customers

Customer deleted successfully!

Delete Customer

Enter Customer ID

Delete Customer

Customer Records

Customer ID	Name	Phone	Email	Address
1	Rahul Sharma	9876543210	rahul@gmail.com	Hyderabad
2	Ananya Reddy	9123456789	ananya@gmail.com	Secunderabad
3	Arjun Kumar	9988776655	arjun@gmail.com	Hyderabad

Implementation Details

Technology Stack

The Vehicle Service Centre Management System is implemented using the following technologies:

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js with Express.js
- Database: MySQL
- Database Hosting: Aiven MySQL
- Database Connectivity: MySQL2 Node.js package
- Development Environment: Visual Studio Code

Database Connectivity:

The Node.js backend connects the web application to the MySQL database using the MySQL2 package. Database credentials are stored securely in a .env file.

Testing Result

- All major test cases were successfully executed. The application correctly performs database connectivity, insertion, viewing, and deletion operations through the web-based UI, and the changes are reflected in the MySQL database.

Conclusion and Future Enhancements

Conclusion

- The Vehicle Service Centre Management System successfully provides a structured and efficient way to manage vehicle service centre data. The system uses a relational MySQL database to manage customers, vehicles, mechanics, service bookings, job cards, spare parts, invoices, and payments. A web-based interface developed using HTML, CSS, and JavaScript, along with Node.js and Express.js, enables users to interact with the database. The implemented system successfully demonstrates insertion, viewing, and deletion of customer records with changes reflected in the database.

Future Enhancements

The system can be further enhanced by adding:

- Online service booking for customers.
- Customer login and registration.
- Service history tracking for each vehicle.
- Automated service reminders and notifications.
- Inventory alerts when spare-part stock is low.
- Online payment and payment gateway integration.
- Automatic invoice generation.
- Mechanic dashboard for managing assigned jobs.
- Reports and analytics for service centre performance.

- Mobile-friendly interface for easier access.

References

- **MySQL Documentation** – MySQL 8.0 Reference Manual. MySQL Documentation
- **Node.js Documentation** – Node.js Official Documentation. Node.js Documentation
- **Express.js Documentation** – Express.js Official Documentation. Express.js Documentation
- **Aiven Documentation** – Aiven for MySQL Documentation. Aiven Documentation
- **MDN Web Docs** – HTML, CSS and JavaScript Web Development Reference. MDN Web Docs
- **DBMS Course Project Guidelines** – Vehicle Service Centre Management System project requirements and evaluation guidelines.

Appendix: GitHub repository link

<https://github.com/Sahitya2305/DBMS-Course-Project>